Week 6 Lecture Notes: Transformers

Hong Qin, Old Dominion University

Fall 2026

Contents

1 Topic 1: Self-Attention and Sequence Representation **2**
1.1 Plain-English Summary . . . 2
1.2 From LSTMs to Transformers: the Role of Attention . . . 2
1.3 Understanding the Components: Queries, Keys, and Values . . . 3
1.4 Scaled Dot-Product Attention . . . 4
1.5 Causal Masking: How Attention Becomes Autoregressive . . . 5
1.6 Multi-Head Attention and the Output Projection . . . 5
1.7 Fixed vs. Variable Dimensions, and the Computational Cost . . . 6
1.8 Worked Toy Example: $T = 3$ . . . 6
1.9 Worked Toy Example: $T = 5$ . . . 6
1.10 Matrix Operations Refresher . . . 7
1.11 Payoff . . . 8

2 Topic 2: Tokenization, Embeddings, and Context Windows **9**
2.1 Plain-English Summary . . . 9
2.2 Tokenization: Turning Text into Integers . . . 9
2.3 Token Embeddings: Lookup Tables . . . 9
2.4 Why Position Must Be Injected . . . 10
2.5 Classical Sinusoidal Positional Encoding . . . 10
2.6 Learned Positional Embeddings . . . 11
2.7 Rotary Position Encoding (RoPE) . . . 11
2.8 Comparison: Additive vs. Rotational Encodings . . . 12
2.9 Context Windows and the $O(N^2)$ Problem . . . 12
2.10 Payoff . . . 13

3 Topic 3: Transformer Layers, Training Intuition, and Inference Behavior **14**
3.1 Plain-English Summary . . . 14
3.2 The Transformer Block . . . 14
3.3 Encoder and Decoder Stacks . . . 14
3.4 Three Architectural Variants . . . 15
3.5 From Hidden State to Next Token: Logits and Softmax . . . 16
3.6 Training Intuition . . . 18
3.7 Inference Pipeline (Decoder-Only, Causal) . . . 18
3.8 Dimensions Cheat Sheet . . . 19
3.9 Interpreting the Output Matrices, and Where "Knowledge" Is Stored . . . 19
3.10 Scaling Laws: Why Bigger Keeps Working . . . 19

3.11 Key Takeaways 20

4 Topic 4: Practical Use of Large-Model Workflows in Applied Settings 21
4.1 Plain-English Summary 21
4.2 The Pre-Trained Model Ecosystem 21
4.3 Four Ways to Adapt a Pre-Trained Model 21
4.4 Hosted APIs vs. Self-Hosting 22
4.5 Agent Frameworks (Forward Reference) 22
4.6 Deployment Considerations 23
4.7 Payoff 23

1 Topic 1: Self-Attention and Sequence Representation

1.1 Plain-English Summary

Module 4 covered RNN-style autoregressive models in which each position is processed in turn, threading a hidden state forward through time. The transformer's defining move is to drop recurrence entirely and instead let every position in a sequence directly look at every other position through a mechanism called **self-attention**. This single change unlocked deeper networks, longer contexts, and the scale that produced modern large language models. The price is a computational cost that grows with the square of the sequence length, but for sequences shorter than a few tens of thousands of tokens it is the right tradeoff.

The transformer architecture splits into an encoder and a decoder, and while both use the scaled dot-product attention mechanism, they differ in how and where this mechanism is applied. This Topic covers the attention operation itself; Topic 3 returns to the encoder and decoder stacks that are built out of it.

1.2 From LSTMs to Transformers: the Role of Attention

Recurrent neural networks (RNNs) and their variants, such as Long Short-Term Memory (LSTM) networks, were the dominant architectures for sequence modeling before the Transformer. This subsection traces the progression from RNNs to LSTMs, to attention-augmented RNNs, and finally to the Transformer.

RNNs. A vanilla RNN maintains a hidden state h_t that is updated recurrently:

$$h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \tag{1}$$

where x_t is the input at time t, W_{xh} and W_{hh} are learnable weight matrices, b_h is a bias, and $\phi(\cdot)$ is a non-linear activation. While simple, RNNs suffer from vanishing and exploding gradients when modeling long sequences.

LSTMs. LSTMs introduced gating mechanisms to address this problem by explicitly controlling information flow:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \tag{2}$$
$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \tag{3}$$
$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c), \tag{4}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \tag{5}$$
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \tag{6}$$
$$h_t = o_t \odot \tanh(c_t), \tag{7}$$

where f_t, i_t, o_t are the forget, input, and output gates, and c_t is the cell state. This design enables LSTMs to capture longer dependencies, but computations remain inherently sequential.

Attention-Augmented LSTMs. To overcome the bottleneck of compressing long sequences into a single hidden state, attention mechanisms were introduced (Bahdanau, Cho, and Bengio,

"Neural Machine Translation by Jointly Learning to Align and Translate," ICLR 2015, arXiv:1409.0473). Given hidden states $\{h_1, \ldots, h_T\}$, attention computes a context vector as a weighted sum:

$$e_{t,s} = v^\top \tanh(W_1 h_{t-1} + W_2 h_s), \tag{8}$$

$$\alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_{k=1}^{T} \exp(e_{t,k})}, \tag{9}$$

$$c_t = \sum_{s=1}^{T} \alpha_{t,s} h_s. \tag{10}$$

The context c_t allows the model to dynamically focus on relevant past states, bypassing the fixed-size bottleneck of LSTMs.

Transformers. The Transformer (Vaswani et al., "Attention Is All You Need," NeurIPS 2017, arXiv:1706.03762) eliminates recurrence altogether, relying solely on self-attention to model dependencies across the entire sequence in parallel. For a sequence of input embeddings $\{x_1, \ldots, x_T\}$, self-attention operates on query, key, and value projections:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V, \tag{11}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \tag{12}$$

where d_k is the dimension of the keys. This mechanism enables each position to attend to all others simultaneously, achieving both long-range dependency modeling and efficient parallel computation.

Summary. Thus, the evolution from RNNs to Transformers can be seen as a gradual shift:

$$\text{RNN} \longrightarrow \text{LSTM} \longrightarrow \text{LSTM + Attention} \longrightarrow \text{Transformer}.$$

Transformers can be interpreted as removing the recurrence entirely and elevating attention to the central modeling primitive.

1.3 Understanding the Components: Queries, Keys, and Values

In the transformer model, the input is projected into three different representations:

- **Queries (Q):** The query vector represents the element for which we are seeking contextual information. In the encoder, each token generates its own query. In the decoder, queries are derived from the previously generated outputs.
- **Keys (K):** The key vectors are used to measure the similarity or relevance of each token relative to the query. Each token in the input sequence is associated with a key that holds essential features of that token.
- **Values (V):** The value vectors contain the actual information that is aggregated. After computing attention weights by comparing queries with keys, these weights are applied to the value vectors to produce the final output of the attention mechanism.

The interaction between queries and keys is central to the attention mechanism. The dot product between a query and all keys measures how well each key matches the query. These scores are then scaled, normalized via a softmax function to generate attention weights, and used to combine the corresponding value vectors. This allows the model to focus on the most relevant parts of the input.

Notation. Let a tokenized prompt of length T be represented by embeddings

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_T^\top \end{bmatrix} \in \mathbb{R}^{T \times d_{\text{model}}}, \quad \mathbf{x}_t \in \mathbb{R}^{d_{\text{model}}}.$$

Standard symbols:

- d_{model}: hidden size of the Transformer block.
- H: number of attention heads.
- d_k, d_v: per-head query/key and value dimensions (often $d_k = d_v = d_{\text{model}}/H$).
- $|V|$: vocabulary size.

For each head (suppressing the head index for clarity) the model learns linear projections

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V,$$

with dimensions

$$\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad \mathbf{W}^V \in \mathbb{R}^{d_{\text{model}} \times d_v},$$

so that $\mathbf{Q} \in \mathbb{R}^{T \times d_k}$, $\mathbf{K} \in \mathbb{R}^{T \times d_k}$, $\mathbf{V} \in \mathbb{R}^{T \times d_v}$.

1.4 Scaled Dot-Product Attention

At the core of both the encoder and the decoder is the scaled dot-product attention defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

where Q (queries), K (keys), V (values), and d_k is the dimensionality of the keys.

What each piece is doing, in plain language.

- $QK^\top$ is a matrix of *similarity scores*: entry (i, j) is the dot product of query q_i with key k_j, measuring how relevant position j is to position i.
- Dividing by $\sqrt{d_k}$ keeps those scores at a reasonable scale so that the next step does not saturate. Without this, large dot products would push the softmax into regions with vanishing gradients.
- softmax along each row turns the similarity scores into a probability distribution over positions: how much attention does position i pay to position j?
- Multiplying by V takes a weighted sum of values, using those attention weights. Position i's output is a mixture of all positions' values, weighted by how relevant they are to i.

You can think of attention as a *differentiable lookup table*: queries probe the keys; relevance scores route information from values back to the query positions.

Per-position form with a causal mask. For position t, compute attention scores to all positions $j \le t$ (causal mask):

$$s_{tj} = \frac{\mathbf{Q}_t \cdot \mathbf{K}_j}{\sqrt{d_k}} + \text{mask}(t, j), \tag{13}$$

$$\alpha_{tj} = \frac{\exp(s_{tj})}{\sum_{m \le t} \exp(s_{tm})}, \qquad \text{(row-wise softmax)} \tag{14}$$

$$\mathbf{z}_t^{(\text{head})} = \sum_{j \le t} \alpha_{tj} \, \mathbf{V}_j. \tag{15}$$

Intuitively: $\mathbf{Q}_t$ asks "what do I need?", $\mathbf{K}_j$ says "what do I offer?", and $\mathbf{V}_j$ carries the information payload delivered if position j is attended to.

1.5 Causal Masking: How Attention Becomes Autoregressive

The basic attention operation has each position attend to every other position. For autoregressive generation (like GPT), position i must only see positions $j \le i$; otherwise the model would "cheat" by looking at the future during training. The standard trick is to add a mask M to the scores before the softmax. This mask sets the upper triangular part of the attention matrix to a very negative value (conceptually $-\infty$), so that the softmax zeroes out any attention weights corresponding to future tokens. Formally:

$$\text{MaskedAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V, \qquad M_{ij} = \begin{cases} 0 & j \le i, \\ -\infty & j > i. \end{cases}$$

This is the only architectural difference between an encoder transformer (which uses bidirectional attention) and a decoder transformer (which uses causal attention). Topic 3 returns to this distinction.

1.6 Multi-Head Attention and the Output Projection

One attention operation gives the model one "way" of attending. Multi-head attention runs H independent attention operations in parallel, each with its own learned projection matrices, then concatenates the results and applies a final linear projection. With H heads, we compute $\mathbf{z}_t^{(h)} \in \mathbb{R}^{d_v}$ per head and concatenate:

$$\mathbf{z}_t^{\text{concat}} = \left[\mathbf{z}_t^{(1)}; \ldots; \mathbf{z}_t^{(H)}\right] \in \mathbb{R}^{Hd_v}.$$

A learned output projection $\mathbf{W}^O \in \mathbb{R}^{(Hd_v) \times d_{\text{model}}}$ maps back to model space:

$$\mathbf{h}_t = \mathbf{z}_t^{\text{concat}} \mathbf{W}^O \in \mathbb{R}^{d_{\text{model}}}.$$

This keeps the block's output dimension equal to its input dimension, enabling residual connections and stacking. In matrix form, over all positions at once,

$$\text{head}_h \;=\; \text{Attention}\Big(XW_h^Q,\, XW_h^K,\, XW_h^V\Big), \qquad \text{MultiHead}(X) \;=\; \text{Concat}(\text{head}_1, \ldots, \text{head}_H)\, W^O.$$

Each head can specialize on a different kind of relationship (one head might track subject-verb agreement; another might track coreference; another might track positional offset). Empirically, models with $H = 8$ to 16 heads behave much better than single-head attention.

1.7 Fixed vs. Variable Dimensions, and the Computational Cost

Fixed per token. The per-token vector sizes are fixed by design:

$$\mathbf{x}_t \in \mathbb{R}^{d_{\text{model}}}, \quad \mathbf{Q}_t, \mathbf{K}_t \in \mathbb{R}^{d_k}, \quad \mathbf{V}_t \in \mathbb{R}^{d_v}.$$

Variable sequence length. The prompt length T scales the attention matrix:

$$\underbrace{\mathbf{Q}}_{T\times d_k} \underbrace{\mathbf{K}^\top}_{d_k\times T} \Rightarrow \underbrace{\mathbf{A}}_{T\times T},$$

so compute and memory scale roughly as $\mathcal{O}(T^2)$. Models also enforce a maximum context length $T_{\max}$ (beyond which the oldest tokens are dropped or summarized).

For modest sequences (a few thousand tokens) the quadratic cost is fine on a single GPU. For very long contexts (32k, 128k, 1M tokens) the quadratic scaling becomes the bottleneck, and a whole subfield (FlashAttention, sliding-window attention, sparse attention, state-space models) exists to mitigate it. Topic 2 returns to this when discussing context-window evolution.

1.8 Worked Toy Example: $T = 3$

Consider a single-head setup with $d_{\text{model}} = 4$, $d_k = d_v = 2$, and three tokens (e.g., `[Paris, is, capital]`). Suppose the learned linear maps yield:

$$\mathbf{Q} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}.$$

Then the (unmasked) score matrix is

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}.$$

After scaling by $\sqrt{d_k} = \sqrt{2}$ and row-wise softmax we might obtain (illustrative):

$$\boldsymbol{\alpha} = \begin{bmatrix} 0.42 & 0.15 & 0.42 \\ 0.15 & 0.42 & 0.42 \\ 0.21 & 0.21 & 0.58 \end{bmatrix}.$$

The context vectors are $\mathbf{Z} = \boldsymbol{\alpha}\mathbf{V}$. For the last token ($t = 3$):

$$\mathbf{z}_3 = 0.21(1, 2) + 0.21(3, 4) + 0.58(5, 6) = (3.7,\ 5.4).$$

This $\mathbf{z}_3$ (concatenated across heads and projected by $\mathbf{W}^O$) forms $\mathbf{h}_3$, which the model maps to logits over the entire vocabulary via $\mathbf{W}^{\text{vocab}}$ (Topic 3).

1.9 Worked Toy Example: $T = 5$

Extend the prompt to five tokens (e.g., `[Paris, is, the, capital, of]`). Let

$$\mathbf{Q} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 1 & -1 \\ 0 & 2 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 1 & -1 \\ 0 & 2 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \\ 9 & 10 \end{bmatrix}.$$

For the final position $t = 5$ (token "of"), $\mathbf{Q}_5 = (0, 2)$. Dotting with rows of $\mathbf{K}$ gives scores [0, 2, 2, −2, 4]. After softmax (illustrative):

$$\boldsymbol{\alpha}_{5:} \approx [0.01,\ 0.12,\ 0.12,\ 0.002,\ 0.75].$$

Then

$$\mathbf{z}_5 \approx 0.01(1, 2) + 0.12(3, 4) + 0.12(5, 6) + 0.002(7, 8) + 0.75(9, 10) = (7.6,\ 8.6).$$

Compared to $T = 3$, with $T = 5$ the attention matrix is 5×5, providing more context to weigh (richer but costlier). The resulting $\mathbf{z}_5$ places strong weight on tokens most semantically relevant to predicting a country next.

1.10 Matrix Operations Refresher

Attention is built almost entirely out of matrix products, but the surrounding architecture (LSTM gates, residual sums, elementwise masks) also uses elementwise operations. It is worth being precise about which is which. Some operations are **elementwise** (entry by entry), while others are **structural** (row-by-column rule or block expansion). Four common operations:

- Normal matrix multiplication (AB or $A \cdot B$)
- Hadamard product ($A \odot B$)
- Hadamard sum ($A \oplus B$)
- Kronecker product ($A \otimes B$)

Normal Matrix Multiplication. If $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, then

$$C = AB \in \mathbb{R}^{m \times p}, \quad C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}.$$

Intuition: Each entry of C is the dot product of a row of A with a column of B. This operation preserves the composition of linear transformations. The score matrix $QK^\top$ and the projections XW^Q are of this kind.

Hadamard Product. If $A, B \in \mathbb{R}^{m \times n}$, then

$$(A \odot B)_{ij} = A_{ij} \cdot B_{ij}.$$

Intuition: Pure elementwise multiplication, like multiplying two images pixel by pixel. The LSTM gate equations above use $\odot$.

Hadamard Sum. If $A, B \in \mathbb{R}^{m \times n}$, then

$$(A \oplus B)_{ij} = A_{ij} + B_{ij}.$$

Intuition: Pure elementwise addition. This is the same as ordinary matrix addition, but the symbol $\oplus$ emphasizes the elementwise operation. Adding the mask M to the score matrix, and adding a residual connection, are of this kind.

Kronecker Product. If $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{p\times q}$, then

$$A \otimes B = \begin{bmatrix} a_{11}B & a_{12}B & \cdots & a_{1n}B \\ a_{21}B & a_{22}B & \cdots & a_{2n}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}B & a_{m2}B & \cdots & a_{mn}B \end{bmatrix} \in \mathbb{R}^{(mp)\times(nq)}.$$

Intuition: Each entry of A is expanded into a block scaled copy of B, producing a much larger block matrix.

Worked Example. Let

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}.$$

Then

$$AB = \begin{bmatrix} 1\cdot 5+2\cdot 7 & 1\cdot 6+2\cdot 8 \\ 3\cdot 5+4\cdot 7 & 3\cdot 6+4\cdot 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}, \qquad A \odot B = \begin{bmatrix} 1\cdot 5 & 2\cdot 6 \\ 3\cdot 7 & 4\cdot 8 \end{bmatrix} = \begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix},$$

$$A \oplus B = \begin{bmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}, \qquad A \otimes B = \begin{bmatrix} 1\cdot B & 2\cdot B \\ 3\cdot B & 4\cdot B \end{bmatrix} = \begin{bmatrix} 5 & 6 & 10 & 12 \\ 7 & 8 & 14 & 16 \\ 15 & 18 & 20 & 24 \\ 21 & 24 & 28 & 32 \end{bmatrix}.$$

Comparison Table.

Symbol	Name	Formula	Output Size	Intuition
AB	Normal multiplication	$C_{ij} = \sum_k A_{ik}B_{kj}$	$m \times p$	Row-by-column dot products, linear map composition
$\odot$	Hadamard product	$(A \odot B)_{ij} = A_{ij}B_{ij}$	$m \times n$	Elementwise multiplication (pixel-by-pixel)
$\oplus$	Hadamard sum	$(A \oplus B)_{ij} = A_{ij} + B_{ij}$	$m \times n$	Elementwise addition
$\otimes$	Kronecker product	Expand each a_{ij} into block $a_{ij}B$	$(mp) \times (nq)$	Block expansion: each entry of A scales entire B

1.11 Payoff

Self-attention replaces the sequential hidden-state chain of an RNN with a single matrix computation: project the sequence to queries, keys, and values; score every pair of positions with a dot product; normalize with a softmax; and mix the values. Every position sees the whole sequence at once, the whole thing is parallel on a GPU, and the only cost is an attention matrix that grows as T^2. Topic 2 covers the things outside the attention operation itself: how text becomes tokens, how tokens become vectors, and how the model knows about position.

2 Topic 2: Tokenization, Embeddings, and Context Windows

2.1 Plain-English Summary

Topic 1 covered the attention operation as if it took vectors as input. Real transformers take *text* as input. This Topic covers the things on either side of attention: how text becomes tokens, how tokens become vectors (token embeddings), how the model knows about position (positional encoding, because attention itself is permutation-invariant), and how the maximum length of input a model can process (the context window) has evolved from 512 tokens in 2017 to over a million tokens in recent models. The engineering for very long context is non-trivial and worth understanding even if you do not implement it yourself.

2.2 Tokenization: Turning Text into Integers

A neural network takes vectors, not strings. The first step is to convert text into a sequence of integer tokens drawn from a fixed vocabulary. Three common strategies:

- **Word-level tokenization**: split on whitespace and punctuation, give each word an integer. Simple, but the vocabulary is huge (English has millions of distinct word forms) and unseen words cannot be handled.
- **Character-level**: each character is a token. Tiny vocabulary, handles anything, but sequences are very long (one token per character).
- **Subword tokenization**: the modern default. Common words become single tokens; rare words split into meaningful pieces. Three popular variants:
  - **Byte-Pair Encoding (BPE)**: used by GPT-2, GPT-3, GPT-4, Llama. Build the vocabulary by iteratively merging the most frequent adjacent character pairs (Sennrich, Haddow, and Birch, ACL 2016, arXiv:1508.07909).
  - **WordPiece**: used by BERT. Similar to BPE but with a likelihood-based merging criterion.
  - **SentencePiece**: used by T5 and Llama. Operates on raw text without a pre-tokenization step and supports many languages cleanly.

Subword tokenization is the right balance: vocabularies stay small (about 32k to 128k tokens), every input string can be tokenized, and common words are still single tokens. The tokenizer is part of the model; you must use the same tokenizer at inference that was used at training.

2.3 Token Embeddings: Lookup Tables

Once a string has been turned into a sequence of integer token ids $(x_1, x_2, \ldots, x_N)$, the model maps each integer to a learnable vector by a lookup table. If the vocabulary has $|V|$ tokens and the model has dimension d_{model}, the embedding table is a learnable $|V| \times d_{\text{model}}$ matrix E. Token x_i becomes the x_i-th row of that matrix:

$$\mathbf{e}_i \;=\; E[x_i, :] \;\in\; \mathbb{R}^{d_{\text{model}}}.$$

Embedding parameters are a non-trivial fraction of total model size (for example, a 50k vocabulary at $d_{\text{model}} = 768$ is about 38M parameters, before any transformer blocks are added). Some implementations tie this input embedding matrix to the output vocabulary projection $\mathbf{W}^{\text{vocab}}$ of Topic 3.

2.4 Why Position Must Be Injected

Transformers treat input sequences as sets of vectors, and self-attention is inherently permutation-invariant. For a sequence matrix X,

$$\text{Attn}(X) = \text{softmax}\left(\frac{XW_Q(XW_K)^\top}{\sqrt{d_k}}\right)XW_V.$$

If we permute the rows of X, the output simply permutes in the same way. Thus, Transformers require *positional encoding* to inject information about the order of tokens. Two major forms are:

1) Additive sinusoidal positional encoding.

2) Rotary positional encoding (RoPE), which encodes position via rotations in 2D subspaces.

A third, learned positional embeddings, sits between them in practice and is covered briefly below.

2.5 Classical Sinusoidal Positional Encoding

Let the embedding dimension be d_{model} and position index be $p \in \{0, \ldots, L-1\}$. We form the input to the Transformer as:

$$X_p = E_p + \text{PE}(p),$$

where E_p is the token embedding and $\text{PE}(p)$ is a deterministic vector.

Sinusoidal Formula. Dimensions are paired: $(2k, 2k+1)$ for $k = 0, \ldots, d_{\text{model}}/2 - 1$. Define frequency:

$$\omega_k = 10000^{-2k/d_{\text{model}}}.$$

Then the sinusoidal embedding is:

$$\text{PE}(p, 2k) = \sin(\omega_k p), \qquad \text{PE}(p, 2k+1) = \cos(\omega_k p).$$

This produces a high-dimensional representation of absolute position. The properties of sine and cosine allow linear layers to recover relative shifts. In plain English: encode position with sinusoids of geometrically spaced frequencies. The clever property is that relative positions can be recovered from linear combinations of these encodings, so the model can learn to attend by relative offset. The encoding is fixed (not learned) and generalizes to longer sequences than seen during training.

Effect in Attention. Queries, keys, and values become:

$$Q_p = X_p W_Q, \qquad K_p = X_p W_K, \qquad V_p = X_p W_V.$$

The attention score between positions i and j is:

$$Q_i \cdot K_j = (E_i + \text{PE}(i))W_Q \cdot (E_j + \text{PE}(j))W_K.$$

Thus, positional information is injected *additively* and separately into each position. This encodes absolute position; relative position is not built in.

2.6 Learned Positional Embeddings

An alternative is an extra $N_{\max} \times d_{\text{model}}$ learnable lookup table, indexed by position, added to the token embedding in the same way as the sinusoidal vector. This is what GPT-2 and many subsequent models use. It is simpler than the sinusoidal scheme but does not generalize beyond the maximum training length $N_{\max}$.

2.7 Rotary Position Encoding (RoPE)

RoPE (Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding," 2021, arXiv:2104.09864) encodes position by applying a rotation to each query and key vector in multiple 2D planes of the embedding. It is used by Llama, GPT-NeoX, DeepSeek, and most 2024 to 2026 open-weights models.

2D Rotation Basics. A rotation by angle θ is:

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}.$$

Key identity:

$$R(\alpha)^\top R(\beta) = R(\beta - \alpha).$$

Applying RoPE to Queries and Keys. Let d be the head dimension and assume d is even. Split a query at position n into 2D blocks:

$$q_n = [\, q^{(x)}_{n,0}, q^{(y)}_{n,0},\ \ldots,\ q^{(x)}_{n,d/2-1}, q^{(y)}_{n,d/2-1} \,].$$

Define the same frequency scheme:

$$\omega_k = 10000^{-2k/d}.$$

For position n, define angle:

$$\theta_{n,k} = n\omega_k.$$

RoPE transforms each 2D pair via:

$$\begin{bmatrix} q^{(x)\prime}_{n,k} \\ q^{(y)\prime}_{n,k} \end{bmatrix} = R(\theta_{n,k}) \begin{bmatrix} q^{(x)}_{n,k} \\ q^{(y)}_{n,k} \end{bmatrix}, \qquad \begin{bmatrix} k^{(x)\prime}_{n,k} \\ k^{(y)\prime}_{n,k} \end{bmatrix} = R(\theta_{n,k}) \begin{bmatrix} k^{(x)}_{n,k} \\ k^{(y)}_{n,k} \end{bmatrix}.$$

Thus, RoPE is a position-dependent **rotation** of queries and keys, without modifying the input embeddings.

Relative Position Emerges Naturally. Consider positions m and n. Take a single 2D block:

$$v_m = \begin{bmatrix} q^{(x)}_{m,k} \\ q^{(y)}_{m,k} \end{bmatrix}, \qquad u_n = \begin{bmatrix} k^{(x)}_{n,k} \\ k^{(y)}_{n,k} \end{bmatrix}.$$

After rotation:

$$v'_m = R(\theta_{m,k}) v_m, \qquad u'_n = R(\theta_{n,k}) u_n.$$

Dot product:

$$(v'_m)^\top u'_n = v_m^\top R(\theta_{m,k})^\top R(\theta_{n,k}) u_n = v_m^\top R(\theta_{n,k} - \theta_{m,k}) u_n.$$

Since $\theta_{n,k} - \theta_{m,k} = (n-m)\omega_k$,

$$(v'_m)^\top u'_n = v_m^\top R((n-m)\omega_k)\, u_n.$$

Thus attention depends directly on the relative position $(n-m)$. Summing over all 2D blocks:

$$\text{score}(m,n) = \sum_k v_m^\top R((n-m)\omega_k) u_n.$$

Hence, RoPE provides translation equivariance.

2.8 Comparison: Additive vs. Rotational Encodings

- **Sinusoidal PE** adds PE(p) to embeddings.
- **RoPE** rotates the query/key vectors in 2D planes using an angle proportional to position.

Both use 2D structure (sin/cos pairs) internally, but in fundamentally different ways:

- Sinusoidal PE uses sin/cos values directly inside the embedding vector.
- RoPE uses sin/cos to parameterize a rotation operator applied inside attention.

Thus the fundamental distinction is:

Addition (absolute) vs. Rotation (relative).

2.9 Context Windows and the $O(N^2)$ Problem

The **context window** is the maximum sequence length the model can process at once. Its historical evolution:

Model	Year	Context window (tokens)
Original transformer	2017	512
GPT-2	2019	1,024
GPT-3	2020	2,048
GPT-4 (initial)	2023	8,192
GPT-4-Turbo	2023	128,000
Claude 3	2024	200,000
Gemini 1.5 Pro	2024	1,000,000+

The headline obstacle to longer context is the $O(N^2)$ memory cost of the attention matrix (Topic 1). Production systems use several techniques to push past it:

- **FlashAttention** (Dao et al., NeurIPS 2022, arXiv:2205.14135): a hardware-aware reimplementation that computes attention without materializing the full $N \times N$ matrix in GPU memory. Same math, much smaller memory footprint.
- **Sliding-window attention**: each position only attends to the nearest w positions. Loses long-range information but cuts cost to $O(N \cdot w)$.
- **Sparse and structured attention**: attend only to a structured subset of positions (for example, a stride pattern).
- **Linear-time alternatives**: state-space models such as Mamba (Gu and Dao, 2023, arXiv: 2312.00752), linear attention variants, recurrent attention. An active research area.

2.10 Payoff

Everything in this Topic sits on either side of the attention operation and is easy to overlook, yet each piece is load-bearing. Tokenization decides what a "position" even is; the embedding table turns integer ids into the vectors that Topic 1 assumed; positional encoding is the only reason a permutation-invariant operation can model word order; and the context window (with the $O(N^2)$ cost behind it) sets the hard limit on how much text the model can consider at once. When you use a pre-trained model in Topic 4, these choices were made for you, and your job is to respect them: use the matching tokenizer, stay inside the context window, and know whether the model uses absolute or rotary positions when you push it past its training length. Topic 3 stacks self-attention into the complete transformer block and walks through how to actually train and run inference on a transformer model.

3 Topic 3: Transformer Layers, Training Intuition, and Inference Behavior

3.1 Plain-English Summary

A transformer model is a stack of identical building blocks called *transformer blocks*, each containing a self-attention layer and a small MLP, wrapped with layer-norm and residual connections. Stack twelve of them and you have GPT-1; stack ninety-six of them at much wider sizes and you have GPT-3. This Topic covers the block structure, the encoder and decoder stacks and their three architectural variants (encoder-only, decoder-only, encoder-decoder) and what each is good for, the readout from hidden state to next-token probabilities (logits and softmax), the training objective (next-token cross-entropy, the same as the Module 4 autoregressive objective but at much larger scale), the inference trick (KV-cache) that makes generation efficient, and the empirical scaling laws that explain why bigger transformer models keep getting better.

3.2 The Transformer Block

One block takes a sequence of vectors of shape $N \times d_{\text{model}}$ as input and produces the same shape as output. It applies, in order:

1. Layer normalization (pre-norm; most modern transformers use this).
2. Multi-head self-attention (causal-masked for decoder transformers).
3. A residual connection adding the input back.
4. Another layer normalization.
5. A position-wise MLP (two linear layers with a non-linearity, expanding then contracting; the inner dimension is typically $4 \cdot d_{\text{model}}$).
6. Another residual connection.

In equations, given input x (shape $N \times d_{\text{model}}$):

$$y \;=\; x \;+\; \text{MultiHead}(\text{LN}(x))\,, \qquad \text{Block}(x) \;=\; y \;+\; \text{MLP}(\text{LN}(y))\,.$$

The residual connections (the $x + \ldots$ and $y + \ldots$ patterns) are what let you stack many blocks without the gradient vanishing. Pre-norm vs. post-norm refers to whether layer-norm is applied before each sublayer (pre-norm, the modern default, more stable training) or after each sublayer plus residual (post-norm, the original 2017 paper).

3.3 Encoder and Decoder Stacks

Encoder. In the **encoder**, every layer performs self-attention on the input sequence. Here, the queries, keys, and values are all derived from the same input x:

$$Q = xW^Q, \quad K = xW^K, \quad V = xW^V.$$

Thus, the encoder's self-attention is computed as:

$$\text{SelfAttention}_{\text{enc}}(x) = \text{softmax}\left(\frac{(xW^Q)(xW^K)^T}{\sqrt{d_k}}\right)(xW^V).$$

This mechanism allows each token in the input to attend to all other tokens, integrating contextual information across the entire sequence.

Decoder. The **decoder** is more complex because it has to generate an output sequence while incorporating information from the encoder. It uses two main attention mechanisms.

1. Masked Self-Attention. The decoder first applies self-attention to its own previous outputs. To maintain the autoregressive property (i.e., ensuring a token only depends on earlier tokens), a mask M is applied. This mask typically sets the upper triangular part of the attention matrix to a very negative value, so that the softmax zeroes out any attention weights corresponding to future tokens. Formally:

$$\text{MaskedAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V.$$

Here, Q, K, and V are derived from the decoder's own input (previously generated tokens).

2. Encoder-Decoder (Cross) Attention. After the masked self-attention, the decoder incorporates information from the encoder. In this step, the queries come from the decoder (from the output of the masked self-attention), while the keys and values come from the encoder's final output. The formula is:

$$\text{EncDecAttention}(Q_{\text{dec}}, K_{\text{enc}}, V_{\text{enc}}) = \text{softmax}\left(\frac{Q_{\text{dec}}K_{\text{enc}}^T}{\sqrt{d_k}}\right)V_{\text{enc}}.$$

This step allows the decoder to "look" at the input sequence and incorporate context from the encoder into the output generation.

Encoder vs. decoder, side by side.

- **Input Sources:**
  - **Encoder:** Uses self-attention where Q, K, and V are all derived from the same input x.
  - **Decoder:** Uses two attention mechanisms: masked self-attention on its own output (with a mask M) and cross-attention that uses the encoder's outputs for K and V.
- **Masking:**
  - **Encoder:** No masking is necessary; all tokens attend to each other.
  - **Decoder:** Uses a mask in the self-attention to prevent future tokens from being attended to, preserving the autoregressive property.
- **Attention Layers:**
  - **Encoder:** A single self-attention layer per encoder block.
  - **Decoder:** Two sequential attention layers (masked self-attention followed by encoder-decoder attention) in each decoder block.

These differences in the attention formulas are key to enabling the decoder to generate coherent output sequences while leveraging the complete context provided by the encoder.

3.4 Three Architectural Variants

Encoder-only (BERT-style). Self-attention is bidirectional (no causal mask): every position attends to every position. Pre-trained on *masked language modeling*: randomly mask out 15% of input tokens and train the model to predict them from context. Used for tasks that require understanding (classification, named-entity recognition, sentence embedding, retrieval). It cannot generate sequentially; it is not a generative model in the way GPT is.

Decoder-only (GPT-style). Self-attention is causal-masked: position i only attends to positions $j \le i$. There is no encoder and therefore no cross-attention layer. Trained as an autoregressive language model: at each position, predict the next token. This is the architecture that became GPT-2, GPT-3, GPT-4, Claude, Llama, and essentially every modern large language model. The vast majority of modern transformer work happens in this variant, and the remainder of this Topic focuses on it.

Encoder-decoder (T5, BART, the original translation transformer). Two transformer stacks, exactly as derived above. The encoder processes the input bidirectionally; the decoder generates the output autoregressively and uses cross-attention to attend to the encoder's output. Used for tasks where input and output are different (machine translation, summarization, conditional generation). Less common in modern LLM work but still important for structured tasks.

3.5 From Hidden State to Next Token: Logits and Softmax

After attention (plus layer norms and the position-wise MLP), the final-layer state $\mathbf{h}_t$ is projected to vocabulary logits:

$$\boldsymbol{\ell}_t = \mathbf{h}_t \, \mathbf{W}^{\text{vocab}} + \mathbf{b} \quad \in \mathbb{R}^{|V|}, \tag{16}$$

$$p(x_{t+1} = v \mid x_{\le t}) = \text{softmax}(\boldsymbol{\ell}_t)_v. \tag{17}$$

Here $\mathbf{W}^{\text{vocab}} \in \mathbb{R}^{d_{\text{model}} \times |V|}$ has one column per vocabulary token, so the output layer *scores the entire vocabulary at once*. Some implementations *tie* $\mathbf{W}^{\text{vocab}}$ with the input embedding matrix; others keep them separate.

The word "logit" and the softmax function deserve a short derivation of their own, because the same construction appears in the LSTM gates of Topic 1, in the attention weights, and in the output layer here.

Binary classification: logit and sigmoid. In binary logistic regression, we model the probability of class 1 using a linear function passed through the sigmoid function:

$$z = \mathbf{w}^\top \mathbf{x} \tag{18}$$

$$p_1 = \sigma(z) = \frac{1}{1 + e^{-z}} \tag{19}$$

$$p_0 = 1 - p_1 = \frac{e^{-z}}{1 + e^{-z}} \tag{20}$$

The logit function (log-odds) is:

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right) = z \tag{21}$$

Reformulating sigmoid as a two-class softmax. To bridge to the softmax function, consider defining logits for both classes:

$$z_1 = \mathbf{w}_1^\top \mathbf{x} \tag{22}$$

$$z_0 = \mathbf{w}_0^\top \mathbf{x} \tag{23}$$

Without loss of generality, we can set $\mathbf{w}_0 = \mathbf{0}$, giving $z_0 = 0$, and $z_1 = z$. The softmax function for class $k \in \{0, 1\}$ is:

$$p_k = \frac{e^{z_k}}{e^{z_0} + e^{z_1}} \tag{24}$$

Substituting:

$$p_1 = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}} = \sigma(z) \tag{25}$$

$$p_0 = \frac{1}{1 + e^z} = 1 - \sigma(z) \tag{26}$$

Conclusion: The sigmoid function is a special case of the softmax function with two logits where one logit is fixed at zero.

Extension to multiclass softmax. For $K > 2$ classes, define a separate score (logit) for each class:

$$z_k = \mathbf{w}_k^\top \mathbf{x} \quad \text{for } k = 1, \ldots, K \tag{27}$$

The softmax function computes the probability of class k as:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \quad \text{for } k = 1, \ldots, K \tag{28}$$

This ensures $0 < p_k < 1$ and $\sum_{k=1}^{K} p_k = 1$. In the language-model output layer, $K = |V|$ and the $\mathbf{w}_k$ are the columns of $\mathbf{W}^{\text{vocab}}$.

Log-odds in the multiclass setting. In the multiclass case, the log-odds between any two classes k and j are:

$$\log\left(\frac{p_k}{p_j}\right) = z_k - z_j \tag{29}$$

This generalizes the binary case, where the log-odds were just z.

Aspect	**Binary (Sigmoid)**	**Multiclass (Softmax)**
Input logits	$z \in \mathbb{R}$	$\mathbf{z} = [z_1, \ldots, z_K] \in \mathbb{R}^K$
Output	One probability p_1	Vector of probabilities $[p_1, \ldots, p_K]$
Function	$\sigma(z) = \frac{1}{1+e^{-z}}$	$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$
Log-odds	$\log\left(\frac{p}{1-p}\right) = z$	$\log\left(\frac{p_k}{p_j}\right) = z_k - z_j$

Final remark. The sigmoid function is a special case of softmax for two classes, where:

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{e^0 + e^z} = \text{softmax}(z_0 = 0, z_1 = z) \tag{30}$$

This illustrates the natural progression from binary classification to multiclass classification within a unified probabilistic framework.

3.6 Training Intuition

Training a decoder-only transformer on a corpus is, mathematically, exactly the same as training the LSTM autoregressive model from Module 4: at each position, predict the next token, and maximize the log-likelihood summed over all tokens in your training set. The training objective is the per-token cross-entropy:

$$\mathcal{L}(\theta) \;=\; -\sum_{i=1}^{N} \log p_\theta(x_i \,|\, x_{<i})\,,$$

where $p_\theta(x_i \mid x_{<i})$ is the softmax output of the previous subsection evaluated at position $i-1$. Because of the causal mask, all N positions can be trained in one parallel forward pass (teacher forcing): the model sees the true prefix at every position and is scored on every next token at once.

The differences from Module 4 are entirely in (a) the architecture (self-attention instead of recurrence) and (b) scale (billions of parameters, trillions of tokens, weeks on thousands of GPUs).

What you can train on a single GPU in a course is a *small* transformer: 12 layers, 768 hidden dimensions, 12 attention heads, about 125M parameters, on a small text corpus, in a few hours. Foster's `gpt` notebook (Chapter 9 of *Generative Deep Learning*, 2nd ed., 2023) does exactly this on a small Keras setup. The architecture is identical to GPT-2; only the scale is different. This is the version you can fully understand and train; the rest of the modern LLM story is "the same thing, much bigger, much more data."

3.7 Inference Pipeline (Decoder-Only, Causal)

At generation time:

1. Tokenize the prompt → embeddings $\mathbf{X}$ (Topic 2).
2. For each position t, compute $\mathbf{Q}_t$ and attend to $\{\mathbf{K}_j, \mathbf{V}_j\}_{j \le t}$ (the causal mask prevents peeking ahead).
3. Concatenate heads → $\mathbf{W}^O$ → residual + MLP → $\mathbf{h}_t$.
4. Project with $\mathbf{W}^{\text{vocab}}$, softmax to get $p(x_{t+1} \mid x_{\le t})$.
5. Sample or argmax the next token, append it to the context, and repeat.

"Knowledge" (facts like *Paris→France*) is not stored solely at the output layer; it is distributed across embeddings, attention projections, and especially the MLPs. The output projection merely reads out whatever representation $\mathbf{h}_t$ encodes.

KV-cache: the inference trick that makes generation tractable. Naive autoregressive generation with a transformer would re-run the entire forward pass over the full prefix at each new token. This is wasteful because the keys and values $\{\mathbf{K}_j, \mathbf{V}_j\}$ for positions you already processed do not change when you add a new token. The **KV-cache** stores the per-layer key and value tensors from previously processed positions and reuses them. Each new token then only requires computing its own query, key, and value and one row of attention against the cached keys and values.

Without the KV-cache, generating an N-token sequence is $O(N^3)$. With the KV-cache, it is $O(N^2)$. This is the difference between "you cannot actually run inference" and "chat assistants work." The size of the cache (layers × heads × context length × d_k) is in turn the main memory bottleneck at long context, which is what the multi-head latent attention of DeepSeek (see the supplementary notes) is designed to shrink.

3.8 Dimensions Cheat Sheet

Quantity	Shape	Purpose
$\mathbf{X}$	$T \times d_{\text{model}}$	Input token embeddings
$\mathbf{W}^Q$	$d_{\text{model}} \times d_k$	Embed to queries per head
$\mathbf{W}^K$	$d_{\text{model}} \times d_k$	Embed to keys per head
$\mathbf{W}^V$	$d_{\text{model}} \times d_v$	Embed to values per head
$\mathbf{Q}, \mathbf{K}$	$T \times d_k$	Queries/keys for all tokens
$\mathbf{V}$	$T \times d_v$	Values for all tokens
Concat heads	$T \times (Hd_v)$	Concatenate H head outputs
$\mathbf{W}^O$	$(Hd_v) \times d_{\text{model}}$	Return to model space
$\mathbf{h}_t$	d_{model}	Hidden state pre-vocab
$\mathbf{W}^{\text{vocab}}$	$d_{\text{model}} \times \lvert V \rvert$	Hidden $\to$ logits over vocab
$\boldsymbol{\ell}_t$	$\lvert V \rvert$	Logits for all tokens

3.9 Interpreting the Output Matrices, and Where "Knowledge" Is Stored

$\mathbf{W}^O$. This matrix merges the H heads back into model space; it is primarily *structural*. It shapes how multiple relational views are combined but does not by itself store a lookup table of facts.

$\mathbf{W}^{\text{vocab}}$. Each column acts like a prototype direction for a vocabulary item. The logit for token v is $\ell_{t,v} = \mathbf{h}_t \cdot \mathbf{w}_v^{\text{vocab}} + b_v$. High alignment means high probability. While $\mathbf{W}^{\text{vocab}}$ defines how to read out words, the factual association (e.g., "Paris $\to$ France") is mainly encoded in the hidden state $\mathbf{h}_t$, which in turn reflects the cumulative effect of embeddings, attention ($\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V, \mathbf{W}^O$), and MLP layers.

Where is knowledge stored? Distributed across:

1. **Embeddings** (distributional semantics).
2. **Attention projections $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V$** (routing patterns: what should query which keys, and what values flow).
3. **Feed-forward (MLP) layers** (most parameters; encode high-level associations and facts).
4. **Output projection $\mathbf{W}^{\text{vocab}}$** (readout interface).

Thus, knowledge is not localized to the final layer; it is a property of the *entire* network.

Mini case: "Paris is the capital of __". At the final prompt position ("of"), the query tends to attend strongly to tokens like "Paris" and "capital" (compare the $T = 5$ toy example in Topic 1). The values from those positions, filtered through learned projections and the MLP, produce a hidden state $\mathbf{h}_t$ aligned with the "France" column of $\mathbf{W}^{\text{vocab}}$, yielding a high logit for the token "France" and thus a high probability after softmax.

3.10 Scaling Laws: Why Bigger Keeps Working

The empirical observation that defined the GPT-3 era: model performance (measured as test loss) follows predictable power laws in model size, dataset size, and compute (Kaplan et al., "Scaling

Laws for Neural Language Models," 2020, arXiv:2001.08361; Hoffmann et al., "Training Compute-Optimal Large Language Models," NeurIPS 2022, arXiv:2203.15556). Concretely:

- For a fixed compute budget, there is an optimal model size and dataset size; doing more of either alone underperforms.
- The Chinchilla scaling law (Hoffmann et al., 2022) found that earlier large models (including GPT-3 itself) were severely under-trained on data; for a given parameter count, you want roughly 20 tokens of training data per parameter.
- Scaling continues to work surprisingly far. Even at the frontier (trillion-parameter models trained on tens of trillions of tokens) the laws still roughly hold; this is why the field keeps scaling up.

3.11 Key Takeaways

- A transformer is a stack of identical blocks: pre-norm, multi-head attention, residual, pre-norm, MLP, residual.
- $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are learned linear projections enabling content-based routing over the sequence; causal masking ensures next-token prediction uses only past context; cross-attention lets a decoder read an encoder.
- $\mathbf{W}^O$ merges multi-head outputs back to model space; $\mathbf{W}^{\text{vocab}}$ scores all tokens at once; the sigmoid of binary logistic regression is the two-class special case of that softmax.
- Training is next-token cross-entropy over the whole corpus, parallel across positions thanks to the mask. Inference is sequential, and the KV-cache is what makes it affordable.
- Per-token dimensions ($d_{\text{model}}, d_k, d_v$) are fixed; only T varies, making attention $T \times T$. Longer prompts increase context richness and quadratic cost; the model's knowledge is distributed across layers.
- Scaling laws say the same architecture keeps improving with more parameters and (proportionally) more data.

4 Topic 4: Practical Use of Large-Model Workflows in Applied Settings

4.1 Plain-English Summary

Most people using transformers today are not training them from scratch. They are using someone else's pre-trained model and adapting it to a specific task or domain. This Topic surveys the modern practical toolkit: where to find pre-trained models (Hugging Face), what your options are for adapting them (prompting, retrieval-augmented generation, fine-tuning, parameter-efficient fine-tuning such as LoRA), when to call a hosted API versus self-host, and how agent frameworks layer task execution on top of base LLMs. This is the workflow most course projects will use; the previous three Topics built the conceptual foundation, and this one is the applied vocabulary.

4.2 The Pre-Trained Model Ecosystem

The single most important resource in modern ML is the Hugging Face Hub (https://huggingface.co/), an open registry of pre-trained models. As of 2026 it hosts over a million models across text, vision, audio, multimodal, and protein domains. Each model has a *model card* documenting the architecture, the training data, the license, the evaluation benchmarks, and known limitations. The `transformers` library (also from Hugging Face) gives you a uniform PyTorch or TensorFlow API for loading and running almost any of them.

A minimal text-generation example:

```
from transformers import AutoModelForCausalLM, AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("gpt2")
model = AutoModelForCausalLM.from_pretrained("gpt2")

inputs = tokenizer("The␣quick␣brown␣fox", return_tensors="pt")
outputs = model.generate(**inputs, max_new_tokens=20,
                         do_sample=True, temperature=0.8)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

A few lines download a 124M-parameter GPT-2 and use it. The same pattern works for billion-parameter models with a bigger machine. Notice that every piece of Topics 1 through 3 is visible here: `AutoTokenizer` is the BPE tokenizer (Topic 2), `generate` runs the causal inference loop with a KV-cache (Topic 3), and `temperature` rescales the logits before the softmax, $p_v \propto \exp(\ell_v/\tau)$, so that lower τ sharpens the distribution and higher τ flattens it.

4.3 Four Ways to Adapt a Pre-Trained Model

1. Prompting (no training). The cheapest option. Write a prompt that elicits the behavior you want. For instruction-tuned models this is often enough. Variations: **zero-shot** (just ask), **few-shot** (provide worked examples in the prompt), and **chain-of-thought** (ask the model to reason step by step before answering; improves performance on math and reasoning tasks). Cost: API tokens or local inference time. No training. Iteration is fast.

2. Retrieval-Augmented Generation (RAG). When the model lacks domain knowledge or is hallucinating facts, give it the relevant context in the prompt. Standard RAG architecture: maintain a vector database of your documents (embedded with a sentence-embedding model); at

query time, retrieve the top-k relevant documents and put them in the LLM's context window along with the user query. The LLM's answer is grounded in retrieved facts rather than its training data. Cost: vector-database infrastructure, embedding inference, and a per-query LLM call. No model training. Standard libraries: LangChain, LlamaIndex, Haystack.

3. Parameter-efficient fine-tuning (PEFT). When prompting and RAG are not enough but you cannot afford full fine-tuning, train only a small number of additional parameters. The dominant technique is **Low-Rank Adaptation (LoRA)** (Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," ICLR 2022, arXiv:2106.09685): freeze the pre-trained weights and add small trainable low-rank update matrices to selected layers (typically the attention projections W^Q, W^V of Topic 1). If the original layer is $W \in \mathbb{R}^{d\times d}$, the LoRA update factorizes the change as

$$W_{\text{adapted}} \;=\; W \;+\; \Delta W \;=\; W \;+\; BA, \qquad B \in \mathbb{R}^{d\times r},\; A \in \mathbb{R}^{r\times d},\; r \ll d,$$

with rank r much smaller than d (typical $r = 8$ to 64). The forward pass becomes $h = xW + xBA$, so the base model is untouched and the adapter can be merged or swapped at will. Training only B and A means roughly 1% as many trainable parameters as full fine-tuning, fits on a single GPU even for large models, and the resulting LoRA weights are tiny (a few MB), so you can keep many task-specific adapters for one base model. The standard library is Hugging Face `peft`. **QLoRA** (Dettmers et al., NeurIPS 2023, arXiv:2305.14314) adds 4-bit quantization of the base model, making it possible to fine-tune even 70B-parameter models on a single 24GB GPU.

4. Full fine-tuning. Update all model weights on your task-specific data using the same next-token cross-entropy as Topic 3. Most expensive, most flexible. Necessary if your domain is far from the pre-training distribution or if you need many task-specific behaviors integrated into one model. For all but the most demanding applications, LoRA is the better starting point.

4.4 Hosted APIs vs. Self-Hosting

Two ways to actually run a pre-trained model in production:

- **Hosted API**: call a vendor endpoint (OpenAI, Anthropic, Google, AWS Bedrock, Together, Replicate). Pay per token. No infrastructure. State-of-the-art models accessible immediately. Tradeoffs: data privacy (your prompts go to a third party), API rate limits, vendor lock-in, and no weight-level fine-tuning on closed models.
- **Self-hosting**: download an open-weights model (Llama, Mistral, Qwen, Gemma, DeepSeek, and others) and run it on your own GPUs. You control the data and the deployment; you pay for hardware and operational complexity. Tools: vLLM, TGI (Text Generation Inference), Ollama, llama.cpp. ODU's research computing clusters are suitable for self-hosting smaller open models (7B to 30B parameters); larger models need multi-GPU setups.

For course projects: hosted APIs are often the right choice for iteration speed. Self-hosting is the right choice if your project explicitly involves the model weights themselves (fine-tuning experiments, mechanistic interpretability, and so on).

4.5 Agent Frameworks (Forward Reference)

An *agent* is an LLM that interacts with tools (APIs, web search, code execution, databases, other LLMs) in a multi-step loop to accomplish a task that requires more than one LLM call. The agent

typically (1) receives a user request, (2) plans the steps needed, (3) calls a tool, (4) observes the result, (5) decides the next step and iterates until done, and (6) returns a final answer. This is the "Agentic AI" territory that the course title points to. We will not cover agents in depth in this course, but for your project: an agent built on top of a strong pre-trained model is often the most natural way to deliver a useful end-to-end system that does something specific in your domain.

4.6 Deployment Considerations

If your project involves an actual user-facing system (even just for a demo), at minimum consider:

- **Latency budget**: what is the acceptable response time? This determines model size, batching, and hardware.
- **Cost per request**: token costs for APIs; amortized hardware costs for self-hosting.
- **Hallucination evaluation**: how do you know when the model is making things up? Critical for any factual application.
- **Safety and content filtering**: what behavior do you not want, and how do you prevent it?
- **Monitoring and logging**: capture queries and outputs (with privacy considerations) so you can diagnose failures and improve the system.

4.7 Payoff

The four adaptation strategies form a ladder of increasing cost: prompting changes nothing in the model, RAG changes only what goes into the context window, LoRA changes a low-rank sliver of the weights, and full fine-tuning changes everything. Start at the bottom of the ladder and climb only when you can name the specific failure that the cheaper option cannot fix. Whichever rung you land on, the mathematics is the same transformer from Topics 1 to 3: the tokenizer and context window bound what the model can see, attention decides what it looks at, and the softmax over $\mathbf{W}^{\text{vocab}}$ is where every answer is finally read out. The Module 6 assignment lets you choose between training Foster's small GPT from scratch (closest to Topic 3) and a Hugging Face pre-trained workflow (closest to this Topic); either path exercises the same machinery.

References

- D. Foster, *Generative Deep Learning*, 2nd ed., O'Reilly, 2023. Chapter 9, "Transformers."
- A. Vaswani et al., "Attention Is All You Need," NeurIPS 2017. arXiv:1706.03762.
- D. Bahdanau, K. Cho, Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," ICLR 2015. arXiv:1409.0473.
- A. Radford et al., "Language Models are Unsupervised Multitask Learners" (GPT-2), OpenAI, 2019.
- T. Brown et al., "Language Models are Few-Shot Learners" (GPT-3), NeurIPS 2020. arXiv: 2005.14165.
- R. Sennrich, B. Haddow, A. Birch, "Neural Machine Translation of Rare Words with Subword Units," ACL 2016. arXiv:1508.07909.
- J. Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding," 2021. arXiv:2104.09864.
- T. Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," NeurIPS 2022. arXiv:2205.14135.
- A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," 2023. arXiv:2312.00752.

- J. Kaplan et al., "Scaling Laws for Neural Language Models," 2020. arXiv:2001.08361.
- J. Hoffmann et al., "Training Compute-Optimal Large Language Models," NeurIPS 2022. arXiv:2203.15556.
- E. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," ICLR 2022. arXiv: 2106.09685.
- T. Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs," NeurIPS 2023. arXiv: 2305.14314.
- S. Rush, "The Annotated Transformer," Harvard NLP, 2018 (updated 2022).
- Hugging Face `transformers` and `peft` documentation, https://huggingface.co/docs.